

Decision Diagrams in Four Hours

CRASH COURSE

How to use this. Every section ends with something to *do* — draw a diagram, run a script, predict a number before you look. Reading it passively will take forty minutes and teach you nothing; doing it will take four hours and leave you able to follow, and check about the decision-diagram side of the project. All the code here has been run; the numbers printed below are the real output. Six files are attached — `bdd.py` and `evdd.py`, about 60 lines each.

Hour 1 — Binary decision diagrams, on paper.....	2
1.1 The one identity everything is built on	2
1.2 Two reduction rules.....	2
1.3 Ordered, reduced, canonical	2
1.4 Do this now	3
Hour 2 — Build one	3
2.1 The ordering experiment	6
2.2 Do this now	7
Hour 3 — Put numbers on the edges	7
3.1 Why sinks don't work.....	7
3.2 The gauge problem, and normalisation	7
3.3 Do this now	8
Hour 4 — The one experiment that explains the project	8
4.1 Do this now	9
Where you now stand.....	Error! Bookmark not defined.

Hour 1 — Binary decision diagrams (BDD), on paper

1.1 The one identity everything is built on

A Boolean function $f(x_1, \dots, x_n)$ can always be split on its first variable:

$$f = (\neg x_1 \cdot f|_{x_1=0}) + (x_1 \cdot f|_{x_1=1})$$

This is the **Shannon expansion**. Apply it recursively and you get a binary **decision tree**: each internal node tests a variable, the left (dashed) edge is "that variable = 0", the right (solid) edge is "= 1", and the leaves are 0 and 1. The tree has 2^n leaves, so as a data structure it is useless. The whole subject is what happens when you compress it.

1.2 Two reduction rules

Compress the tree into a DAG by applying these until neither fires:

1. **Redundant test.** If a node's two children are the same node, the function doesn't depend on that variable here. Delete the node and redirect its incoming edges to the child.
2. **Sharing.** If two nodes test the same variable and have the same pair of children, they represent the same function. Merge them.

That's it. That's the entire compression mechanism. Nothing cleverer is happening anywhere in this field — the sophistication is all in *choosing the variable order*, which is Hour 2.

1.3 Ordered, reduced, canonical

Fix a variable order and require every path to test variables in that order (skipping is allowed, reordering is not). That's an **OBDD**. Apply the two rules to exhaustion and you get the **reduced** OBDD, and then:

Canonicity theorem (Bryant, 1986). For a fixed variable order, every Boolean function has exactly *one* reduced OBDD, up to isomorphism.

The proof is a two-line induction once you see the right statement, and the right statement is the one that matters for your project:

The nodes labelled x_r in the reduced OBDD are in bijection with the distinct non-constant subfunctions $f|_{x_1 \dots x_{r-1}=a}$, as a ranges over all assignments to the earlier variables.

Read that again. A node *is* a subfunction. Two prefixes that leave the same remaining function must reach the same node (rule 2 merges them); two prefixes that leave different functions

cannot (they differ on some completion). The number of nodes at level r — the **width** of that level — is therefore not an artefact of how you built the diagram. It is a property of f and the order.

Canonicity is why decision diagrams are useful: equality testing becomes pointer comparison, and satisfiability becomes "is the root the 0 sink?". Physicists have nothing like this, which is one of the things the project is trying to trade.

1.4 Do this now

On paper, with order $x_1 < x_2 < x_3$:

1. Draw the reduced OBDD of $x_1 \wedge x_2$. (3 nodes: one test each for x_1 and x_2 , plus sinks. The x_3 variable never appears — that's rule 1 doing its job.)
2. Draw $x_1 \oplus x_2 \oplus x_3$. (Width 1, 2, 2 — parity needs to remember only one bit, "even so far or odd so far".)
3. Draw majority-of-3. (4 internal nodes.)
4. Now the important one. Take $(x_1 \wedge x_2) \vee (x_3 \wedge x_4) \vee (x_5 \wedge x_6)$ and draw it twice: once with order $x_1, x_2, x_3, x_4, x_5, x_6$, once with $x_1, x_3, x_5, x_2, x_4, x_6$. Count nodes both times.

Predict the two counts before you draw.

You do not need to finish the second drawing if you see why it explodes — after testing x_1, x_3, x_5 you must remember all three bits to know what to do later, so the level below has 2^3 distinct subfunctions. In the good order you only ever have to remember "have I already found a satisfied pair?", which is one bit.

Hour 2 — Build one (exercise 1.4.4)

Sixty lines. Type it rather than pasting; the typing is where the understanding happens.

Building a **small library for Boolean functions** — one where a function is represented by an integer, and the library guarantees that two functions that are equal get the *same* integer. That guarantee is Bryant's canonicity theorem turned into working code.

Concretely there are two global tables and three functions.

The tables are T, mapping a node id to its triple (variable, low-child, high-child), and H, the reverse map from triple back to id — the "unique table". Ids 0 and 1 are the sinks; everything else is an internal node.

The three functions you write are:

1. mk(var, low, high) — the only way a node is ever created. It applies rule 1 (if the two children are the same, return the child, don't make a node) and rule 2 (if this exact triple already exists in H, return the existing id).

```
T = {} # id -> (var, low, high)      var is 1..n, low = branch for x=0
H = {} # (var, low, high) -> id      the "unique table"
```

```
def mk(var, low, high):
    if low == high:                # rule 1: this test is redundant
        return low
    key = (var, low, high)
    if key in H:                   # rule 2: this node already exists
        return H[key]
    u = len(T) + 2                 # 0 and 1 are taken by the terminals
    T[u] = key; H[key] = u
    return u
```

mk is the whole package. Both reduction rules live in those five lines, and because every node in the system is created through **mk**, the diagram is never un-reduced, not even transiently. This trick is called **hash-consing**, and it is the single most important implementation idea in the field. When in the future (probably a week later) you design your EVDD implementation, the first thing to check is "is every node created through a **mk**-style unique table?" — if the answer is no, their diagrams are not canonical and every size measurement you report is meaningless.

2. var(i) — the atomic building block: **mk(i, 0, 1)**, the diagram for the single variable x_i .

3. apply(op, u, v) — the combinator. Give it two diagrams and AND/OR/XOR, and it returns the diagram of the combined function. Split both on whichever variable comes first in the order, recurse on the four sub-cases, rebuild with **mk**, memoise.

```
def var_of(u, n):
    return n + 1 if u in (0, 1) else T[u][0]

def apply(op, u1, u2, n, memo=None):
    if memo is None: memo = {}
    if (u1, u2) in memo: return memo[(u1, u2)]
    if u1 in (0, 1) and u2 in (0, 1):
        r = op(u1, u2)
    else:
        v = min(var_of(u1, n), var_of(u2, n))
        l1, h1 = (T[u1][1], T[u1][2]) if var_of(u1, n) == v else (u1, u1)
        l2, h2 = (T[u2][1], T[u2][2]) if var_of(u2, n) == v else (u2, u2)
        r = mk(v, apply(op, l1, l2, n, memo), apply(op, h1, h2, n, memo))
    memo[(u1, u2)] = r
    return r

AND = lambda a, b: a & b
OR  = lambda a, b: a | b
XOR = lambda a, b: a ^ b
def var(i): return mk(i, 0, 1)
```

The memo table is not an optimisation, it is the complexity bound: without it `apply` is exponential, with it it is $O(|G_1| \cdot |G_2|)$. That bound — *operations cost the product of the sizes* — is the reason the whole field cares so much about size, and hence about variable order.

Here's an actual trace of building $x_1 \wedge x_2$:

var(1) -> 2 T = {2: (1, 0, 1)}

var(2) -> 3 T = {2: (1, 0, 1), 3: (2, 0, 1)}

AND -> 4 T = {2: (1, 0, 1), 3: (2, 0, 1), 4: (1, 0, 3)}

AND again -> 4 (same id — nothing new was created)

OR -> 5 T = {..., 5: (1, 3, 1)}

Node 4 reads "test x_1 ; on 0 go to sink 0; on 1 go to node 3 (which is x_2)" — that's $x_1 \wedge x_2$ in two internal nodes. Node 5 is "test x_1 ; on 0 go to x_2 ; on 1 go to sink 1" — that's $x_1 \vee x_2$. Notice AND and OR both *reuse* node 3 rather than each making their own copy, and asking for the AND a second time returns the identical id 4. Four nodes total in memory for two functions.

Potential questions:

1. why it splits into the pair of low children, and the pair of high children?

Write $f \text{ op } g$ and fix the top variable v to a value b . Then $(f \text{ op } g)|_{v=b} = (f|_{v=b}) \text{ op } (g|_{v=b})$

The reason is that op is applied **pointwise**: to evaluate $f \text{ op } g$ on some input, you evaluate f on that input and g on *that same input*, then combine. If the input has $v = 0$, then both f and g are being consulted with $v = 0$. There is no input on which f sees $v = 0$ while g sees $v = 1$ — so a pair like $(f_{\text{low}}, g_{\text{high}})$ is the answer to a question nobody asked. Only the diagonal pairs correspond to real inputs.

2. **Why both diagrams split on the same v .** You take $v = \min(\text{top variable of } u_1, \text{top variable of } u_2)$. If one diagram's top variable comes later in the order, that diagram simply doesn't test v , so $f|_{v=0} = f|_{v=1} = f$ — which is the (u, u) line you see instead of taking children. You still split, you just pass the same node down both branches.

3. **Why this needs a shared variable order.** The step only works because "the top variable of u_1 " and "the top variable of u_2 " are comparable — you can name a single v to split both on. If the two diagrams used different orders, there would be no common variable to peel off, and no $O(|G_1| \cdot |G_2|)$ algorithm.

The rest (`size`, `evaluate`, `satcount`, `width`) is bookkeeping and is in the attached `bdd.py`.

2.1 The ordering experiment

Build $(x_1 \wedge x_2) \vee (x_3 \wedge x_4) \vee \dots$ under a chosen order and count nodes. Real output:

```
n= 6  good order:   6 nodes  bad order:   14 nodes
n= 8  good order:   8 nodes  bad order:   30 nodes
n=10  good order:  10 nodes  bad order:   62 nodes
n=12  good order:  12 nodes  bad order:  126 nodes
```

n versus $2^{(n/2+1)} - 2$. Linear against exponential, from a *permutation of the same variables*, on the same function. For $n = 6$ you can brute-force all 720 orders: the best is 6 nodes, the worst is 14, and the level widths tell you exactly where the blow-up sits —

```
good 1,2,3,4,5,6: size= 6  width=[1, 1, 1, 1, 1, 1]
bad  1,3,5,2,4,6: size=14  width=[1, 2, 4, 4, 2, 1]
```

Why Track B exists.

A permutation of the variables is one object with two names. Decision-diagram people call it the variable order and measure node count. Tensor people call it the site order and measure bond dimension. Level r of the diagram is bond r of the tensor train — so a single permutation fixes both numbers at once.

On the pairs function $(y_1 \wedge y_2) \vee (y_3 \wedge y_4) \vee \dots$ at $n=12$: (bond_vs_nodes.py)

order	DD nodes	max bond dim	numbers a tensor train stores
1,2,...,12	12	3	120
1,3,...,11,2,4,...,12	126	63	10 792

Same function, one permutation changed. The diagram gets 10× bigger; the tensor train gets **90×** bigger, because storage goes like χ^2 .

The decision-diagram community has a mature heuristic for choosing this order by **sifting**: slide one variable through every position with cheap adjacent swaps, keep the best spot, repeat. The tensor community has no equivalent, despite paying a steeper price for a bad order. Track B is porting sifting across. The search space is literally the same $n!$ permutations, so you keep the search strategy and swap the score (node count $\rightarrow$ bond dimension). The adjacent-swap move already exists on the tensor side: contract two neighbouring cores, permute the physical indices, re-split by SVD.

The open question: node count and bond dimension are different functions of the same knob. Does the permutation that minimises one also minimise the other? Nobody has checked.

2.2 Do this now

Run `test_bdd.py`. Then, before running anything else, predict: what is the reduced OBDD size of the parity function on 4 variables, under *any* order? (7 nodes, width 1,2,2,2 — parity is the standard example of a function whose size is order-independent. Contrast with the previous experiment and you have the two extremes.)

Hour 3 — Put numbers on the edges

3.1 Why sinks don't work

The project is about quantum states: $f : \{0,1\}^n \rightarrow \mathbb{C}$, an amplitude per basis state. The naive fix to a BDD is to allow many sinks, one per value — that's an **ADD** or **MTBDD**. It is useless here, because a generic quantum state has 2^n distinct amplitudes and therefore 2^n sinks.

The fix is to put the numbers on the **edges** and multiply along the path:

$f(\alpha) = \prod w(e)$ over the edges e on the path that α selects
Now n weighted edges encode one amplitude as a product of n numbers, and — this is the point — completely different amplitudes can share almost all of their path. This is an **edge-valued decision diagram (EVDD)**, and it is what the MQT quantum simulators actually run on.

3.2 The gauge problem, and normalisation

Edge weights are not unique: multiply one edge by c and the edges below it by $1/c$ and f is unchanged. Left alone, this destroys canonicity — the same function would have infinitely many diagrams. So you fix the freedom by a **normalisation rule**. The simplest: divide both of a node's outgoing weights by the first non-zero one, and push the discarded factor up the incoming edge.

```
def mknode(var, e0, e1):
    """Return a NORMALISED edge (c, node) for the node with children e0, e1."""
    (w0, n0), (w1, n1) = e0, e1
    if w0 == 0 and w1 == 0:
        return (0, ZERO)
    c = w0 if w0 != 0 else w1          # normalisation: first non-zero weight -> 1
    e0, e1 = (w0 / c, n0), (w1 / c, n1)
    if e0 == e1:                      # rule 1: redundant test
        return (c, e0[1])
    return (c, _hash(var, e0, e1))    # rule 2 lives in _hash (sharing)
```

Compare with `mk` from Hour 2: identical shape, one extra line for the normalisation. That is genuinely all the difference between a BDD and an EVDD.

It is the crux of Track A. That gauge freedom is *the same freedom* a tensor train has on its bonds. Decision diagrams fix it by the normalisation above; the tensor community fixes it by

orthogonalisation, because SVD truncation is only optimal in the orthogonal gauge. Two different fixings of one freedom.

3.3 Do this now

Run `test_evdd.py`, which builds a few small states directly from their amplitude tables. Real output for 4 qubits:

```
GHZ          size= 7 width=[1, 2, 2, 2]
W            size= 7 width=[1, 2, 2, 2]
uniform      size= 0 width=[0, 0, 0, 0]
IQP-ish      size= 6 width=[1, 2, 2, 1]
```

Three things to notice, in increasing order of importance.

The uniform superposition has **zero internal nodes** — every test was redundant, and the whole state collapsed to a single weighted edge. That is what a product state looks like: no correlation between qubits, nothing to remember, bond dimension 1.

GHZ has **width 2 everywhere**. One bit to remember: "am I on the all-zeros branch or the all-ones branch?" The MPS bond dimension of GHZ is also 2. That is not a coincidence.

And the same state under a different qubit order has a different size — the little experiment at the end of `test_evdd.py` gives min 6, max 9 for a four-qubit state. The ordering problem does not go away when you add weights.

Hour 4 —one experiment that explains the project

Everything so far has been setup. This is the payoff, and it takes ten minutes.

Set-up. Fix a cut of the variables into a prefix x_1x_2 and a suffix x_3x_4 , and lay the function out as a matrix M : one row per assignment to the prefix, one column per assignment to the suffix. Row a of M is precisely the subfunction $f|_{x_1x_2=a}$ from §1.3.

Now the two quantities:

- **The OBDD/EVDD width at that level** = the number of **distinct rows** of M .
- **The tensor-train bond dimension at that cut** = the **rank** of M .

Take M with rows $[1,0,0,0]$, $[0,1,0,0]$, $[1,1,0,0]$, $[1,-1,0,0]$. Four distinct rows. But the third is the sum of the first two and the fourth is their difference, so the rank is 2. Verified output:

```
distinct rows (= EVDD width at level 3): 4
rank of M (= TT bond dimension there): 2
singular values: [1.7321 1.7321 0. 0.]
```


A decision diagram counts distinct subfunctions. A tensor train counts the dimension of their span. Distinct rows is always at least rank, so the tensor train is never bigger and is sometimes exponentially smaller. What closes the gap is allowing a node to have *several* 0-edges and 1-edges, so that evaluation becomes "sum over all selected paths of the product along each path" — which is exactly what a product of matrices computes. That is the **nondeterministic EVDD**, and *that* is what the bridging paper proves equals a tensor train.

With that one sentence, both tracks become obvious:

- **Track A (truncation).** The singular values above are $[1.73, 1.73, 0, 0]$. Truncation says: when a singular value is small but not zero, throw it away anyway and accept a controlled error. That is a *low-rank approximation of the span* — and notice it is not achievable by merging nodes, because merging can only identify rows that are already equal, whereas SVD first *rotates* into a better basis. The existing approximate-decision-diagram methods delete low-weight subtrees, which is the axis-aligned version. The project's job is to make that comparison precise and then measure it.
- **Track B (ordering).** Change the order and you change which subfunctions sit below the cut, hence both the width and the rank. Sifting is the well-tested procedure for searching that space cheaply on the diagram side. Nobody has run it on the tensor side.

4.1 Do this now

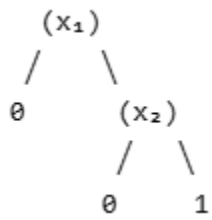
Construct, by hand, a 4-variable function whose width at the middle cut is 4 but whose rank is 3, and check it with the script. Then try to build one where width and rank are equal at every cut. (*Any function whose rows are pairwise linearly independent — e.g. distinct standard basis vectors. That is the case where a decision diagram loses nothing, and it tells you exactly which functions the tensor formalism buys you nothing on.*)

Five questions you should now be able to answer without notes. If any of them is shaky, the fix is in the hour that introduced it, not further reading.

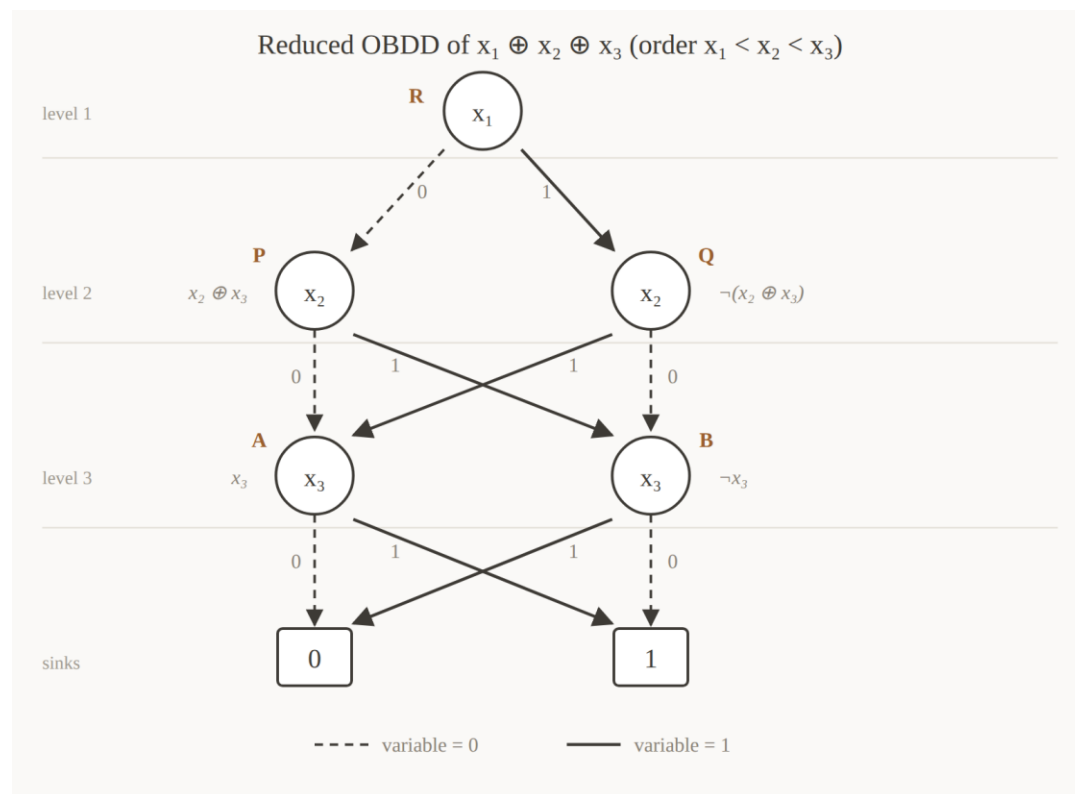
1. Why is a reduced OBDD canonical, given the order?
2. What does a node at level r mean?
3. Why does variable order change the size, and by how much can it?
4. What is the gauge freedom on edge weights, and how is it fixed?
5. Width versus rank — which is bigger, and what feature closes the gap?

Answers:

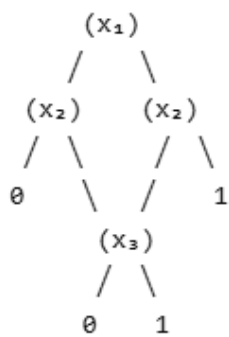
1.4.1



1.4.2



1.4.3



1.4.4

